Data Structures (ETCCDS202)

Unit–1 Assignment: Foundations & Algorithmic Analysis

Course Details

Name of the School: School of Engineering & Technology

Program: B.Tech CSE (AI ML)

Course Title: Data Structures

Course Code: ETCCDS202

Unit Title: Foundations & Algorithmic Analysis

Student Name: Ankit Kumar Mishra

Roll No.: 2501730079

Section: A

Introduction

In this assignment, I implemented different algorithms like Stack ADT, Factorial, Fibonacci, Tower of Hanoi and Binary Search using Python. The main purpose of this assignment was to understand recursion, time complexity and space complexity in a practical way. By writing and testing the code myself, I was able to see how different algorithms behave for different input sizes and why efficiency is important in Data Structures.

Stack ADT (ADT + Operations)

1. What is an ADT?

ADT stands for Abstract Data Type.
It tells us what operations can be performed on data, but it does not tell how the data is stored internally.

For example, Stack ADT allows operations like push, pop, peek, is empty and size.
But internally it can be implemented using a list or array.

In this assignment, I implemented StackADT using a Python list because it is simple and easy to understand.

2. Why Stack Operations are O(1)?

In a stack, we always insert and remove elements from the top.

- push() adds element at the top
- pop() removes element from the top

These operations do not depend on how many elements are inside the stack.
So the time required does not increase when size increases.

Therefore:

Time Complexity = O(1)
Best case = O(1)
Worst case = O(1)
Big-Θ = O(1)

3. Use of Stack in My Program

I used my StackADT to store the moves of Tower of Hanoi.

Every time a disk is moved, the move is pushed into the stack.
At the end, I counted total moves using stack size().

This helped me understand how stack can be used to store recursive traces.

Factorial & Fibonacci (Recursion + Complexity)

1. Recursive Factorial

Factorial of a number n is:

$n! = n \times (n-1) \times (n-2) \times ... \times 1$

Base Case:

If $n = 0$ or $n = 1$ → return 1

Recursive Case:

$factorial(n) = n \times factorial(n-1)$

Complexity of Factorial

Time Complexity:

The function calls itself n times.

So,

Time Complexity = $O(n)$
Big-Ω = $O(n)$
Big-Θ = $O(n)$

Space Complexity:

Each recursive call is stored in memory stack.

Space Complexity = $O(n)$

2. Fibonacci

(a) Naive Recursive Fibonacci

Formula:

$fib(n) = fib(n-1) + fib(n-2)$

Base case:

If $n \leq 1$ → return n

In this method, each function again calls two more functions.

Because of repeated calculations, the number of recursive calls increases very fast.

Time Complexity = $O(2^n)$
Space Complexity = $O(n)$

Big-Ω = $O(2^n)$
Big-Θ = $O(2^n)$

This is why naive Fibonacci becomes very slow for larger values like n = 30.

(b) Memoized Fibonacci (Dynamic Programming)

In this version, I used a dictionary to store already calculated values.

If a Fibonacci number is already calculated, it is directly returned from dictionary.

So repeated work is avoided.

Time Complexity = O(n)
Big-Ω = O(n)
Big-Θ = O(n)

Space Complexity = O(n)

This version is much faster than naive version.

3. Call Counter

I added a global counter in both versions to count how many times function is called.

When I tested:

For n = 30
Naive version had very large number of calls.
Memoized version had very less calls.

This clearly shows why memoization improves efficiency.

Tower of Hanoi

Tower of Hanoi is a classic recursive problem.

Rules:

1. Move only one disk at a time
2. Bigger disk cannot be placed on smaller disk

Why Number of Moves is $2^n - 1$?

To move n disks:

- First move n−1 disks to helper rod
- Then move largest disk
- Then move n−1 disks again

So recurrence becomes:

$T(n) = 2T(n-1) + 1$

When solved, it becomes:

$T(n) = 2^n - 1$

Complexity

Time Complexity = $O(2^n)$
Big-Θ = $O(2^n)$

Space Complexity:

Recursion depth = n

Space Complexity = O(n)

Output for N = 3

Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C

Total moves = 7

Recursive Binary Search

Binary search works only on sorted arrays.

It follows Divide & Conquer approach.

Recurrence Relation

Each time the array is divided into half.

So recurrence is:

$T(n) = T(n/2) + O(1)$

Solving this gives:

Time Complexity = O(log n)

Cases

Best Case:

If element is found in first comparison → O(1)

Average Case:

Element found after few divisions → O(log n)

Worst Case:

Element not present or found at last level → O(log n)

Big-Ω = O(1)
Big-O = O(log n)
Big-Θ = O(log n)

Space Complexity

Since recursion is used,

Recursion depth = log n

Space Complexity = O(log n)

Paradigm Reflection

1. Binary Search – Divide & Conquer

Binary search divides the array into two halves.
It solves only one half recursively.

So it divides the problem and conquers one part.

That is why it is Divide & Conquer.

2. Fibonacci Memoization – Dynamic Programming

Naive Fibonacci repeats same calculations again and again.

Memoization stores results of smaller problems and reuses them.

So repeated work is avoided.

That is why it is Dynamic Programming.

3. Tower of Hanoi – Recursion

Tower of Hanoi solves a big problem by solving smaller subproblems.

Base case:

If only 1 disk → move directly.

Otherwise:

Move n−1 disks, then largest disk, then n−1 disks again.

This shows how recursion reduces problem size step by step.

4. Real Life Examples

Divide & Conquer:

Searching a word in dictionary by opening middle page first.

Dynamic Programming:

Finding shortest path using already calculated distances.

Greedy:

Giving change using largest currency note first.

Conclusion

Through this assignment, I gained a clear understanding of how recursion actually works in programming. Earlier, recursion seemed confusing to me because a function keeps calling itself, but after implementing problems like Factorial, Fibonacci, Tower of Hanoi and Binary Search, I understood how a large problem is reduced into smaller subproblems until the base case is reached.

I also learned the importance of analyzing time and space complexity. By comparing naive Fibonacci with memoized Fibonacci, I clearly saw how repeated calculations can make a program slow and how optimization techniques like memoization can improve performance. Adding the call counter helped me practically observe the difference in number of recursive calls.

Implementing Stack ADT helped me understand the concept of Abstract Data Type and why stack operations usually take constant time. Using the stack to store Tower of Hanoi moves made the learning more practical.

Overall, this assignment improved my logical thinking and helped me understand algorithm efficiency using Big-O notation. It gave me confidence in writing recursive programs and analyzing their performance.